

分布式TensorFlow多主机多GPU原理与实现

转载

Locutus

于 2018-07-08 01:04:40 发布

阅读量1w

收藏 35

点赞数 8

分类专栏:

Tensorflow/Pytorch

Tensorflow/Pytorch 专栏收录该内容

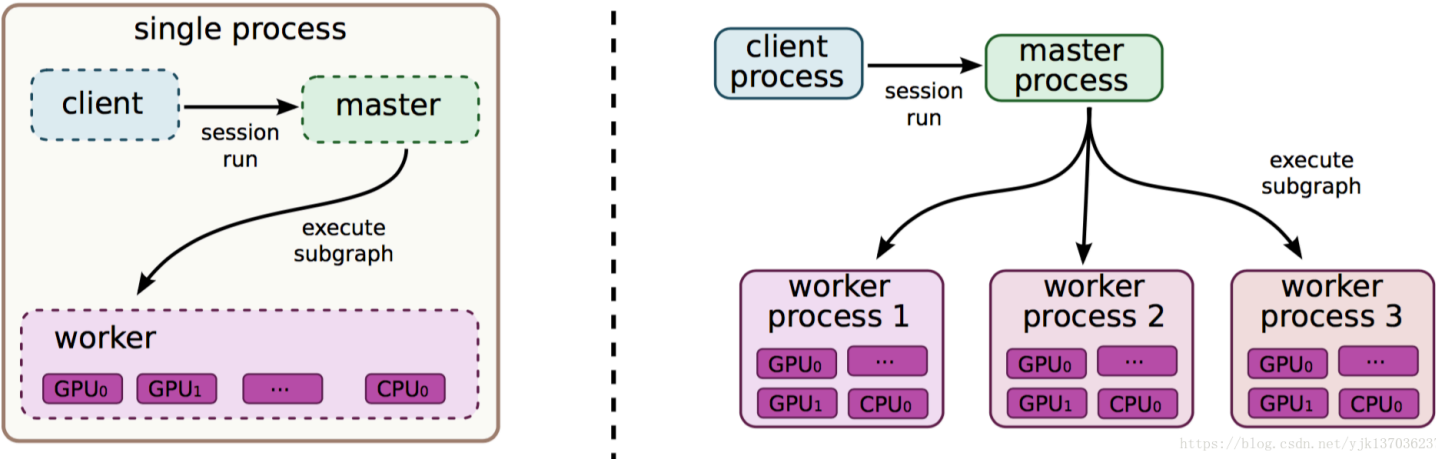
16 篇文章

订阅专栏

 **摘要** 本文详细介绍TensorFlow的单机和分布式部署方法，包括单机多GPU训练、多主机多GPU训练等，并提供了一个手写字体识别的分布式TensorFlow应用实例。

摘要生成于 [C知道](#)，由 DeepSeek-R1 满血版支持，[前往体验](#) >

TF的部署分为单机部署和分布式部署。在训练集数据量很大的情况下，单机跑深度学习程序过于耗时，所以需要分布式并行计算。在分布式部署中，我们需要在不同主机节点，实现client，master，worker。



1. Single-Device Execution

1.1 单机多GPU训练

构建好图后，使用拓扑算法来决定执行哪一个节点，即对每个节点使用一个计数，值表示所依赖的未完成的节点数目，当一个节点的运算完成时，将依赖该节点的所有节点的计数减一。如果节点的计数为0，将其放入准备队列等待执行。

单机的多GPU训练，tensorflow的官方已经给了一个cifar的例子，已经有比较详细的代码和文档介绍，这里大致说下多GPU的过程，以便方便引入到多机多GPU的介绍。单机多GPU的训练过程如下：

- 假设你的机器上有3个GPU
- 在单机单GPU的训练中，数据是一个batch一个batch的训练。在单机多GPU中，数据一次处理3个batch(假设是3个GPU训练)，每个GPU处理一个batch的数据计算
- 变量，或者说参数，保存在CPU节点上
- 刚开始的时候数据由CPU分发给3个GPU，在GPU上完成计算，得到每个batch要更新的梯度
- 然后在CPU上收集完了3个GPU上的要更新的梯度，计算一下平均梯度，然后更新参数。
- 继续循环这个过程。

通过这个过程，处理的速度取决于最慢那个GPU的速度。如果3个GPU的处理速度差不多的话，处理速度就相当于单机单GPU的速度的3倍减去数据在CPU和GPU之间传输的开销，实际的效率提升看CPU和GPU之间数据的速度和处理数据的大小。

1.2 通俗解释

老师给小明和小华布置了10000张纸的乘法题并且把所有的乘法的结果加起来，每张纸上有128道乘法题。这里一张纸就是一个batch，batch_size就是128。小明算加法比较快，小华算乘法比较快，于是小华就负责计算乘法，小明负责把小华的乘法结果加起来。这样小明就是CPU，小华就是GPU。

这样计算的话，预计小明和小华两个人得要花费一个星期的时间才能完成老师布置的题目。于是小明就招来2个算乘法也很快的小红和小亮。于是每次小明就给小华，小红，小亮各分发一张纸，让他们算乘法，他们三个人算完了之后，把结果告诉小明，小明把他们的结果加起来，然后再给他们每人分发一张算乘法的纸，依次循环，知道所有的算完。

限时
立减
¥90

GO>





这里小明采用的是同步模式，就是每次要等他们三个都算完了之后，再统一算加法，算完了加法之后，再给他们三个分发纸张。这样速度就取决于他们三个中算乘法算的最慢的那个人，和分发纸张的速度。

2. Multi-Device Execution

在分布式系统情况下，事情就变得复杂了很多，还好前述调度用了现有框架。那么对于TF来说，剩下的事情就是：

- 决定运算在哪个设备上运行
- 管理设备之间的数据传递

2.1 分布式多主机多GPU训练

随着设计的模型越来越复杂，模型参数越来越多，越来越大，大到什么程度？多到什么程度？多参数的个数上百亿个，训练的数据多到按TB级别来衡量。大家知道每次计算一轮，都要计算梯度，更新参数。当参数的量级上升到百亿量级甚至更大之后，参数的更新的性能都是问题。如果是单机16个GPU，一个step最多也是处理16个batch，这对于上TB级别的数据来说，不知道要训练到什么时候。于是就有了分布式的深度学习训练方法，或者说框架。

- 参数服务器

在介绍tensorflow的分布式训练之前，先说下参数服务器的概念。

前面说道，当你的模型越来越大，模型的参数越来越多，多到模型参数的更新，一台机器的性能都不够的时候，很自然的我们会想到把参数分开放到不同的机器去存储和更新。

因为碰到上面提到的那些问题，所有参数服务器就被单独拎出来，于是就有了参数服务器的概念。参数服务器可以是多台机器组成的集群，这个就有点类似分布式的存储架构了，涉及到数据的同步，一致性等等，一般是key-value的形式，可以理解为一个分布式的key-value内存数据库，然后再加上一些参数更新的操作。详细的细节可以去google一下，这里就不详细说了。反正就是当性能不够的时候，

几百亿的参数分散到不同的机器上去保存和更新，解决参数存储和更新的性能问题。

借用上面的小明算题的例子，小明觉得自己算加法都算不过来了，于是就叫了10个小明过来一起帮忙算。

- gRPC (google remote procedure call)

TensorFlow分布式并行计算基于gRPC通信框架，其中包括一个master创建Session，还有多个worker负责执行计算图中的任务。

gRPC首先是一个RPC，即远程过程调用。通俗的解释是：假设你在本机上执行一段代码num=add(a,b)，它调用了过程 call，然后返回了一个值num，你感觉这段代码只是在本机上执行的，但实际情况，本机上的add方法是将参数打包发送给服务器，然后服务器运行服务器端的add方法，返回的结果再将数据打包返回给客户端。

- 结构

Cluster是Job的集合，Job是Task的集合。即一个Cluster可以切分多个Job，一个Job指一类特定的任务，每个Job包含多个Task，比如parameter server(ps)、worker。在大多数情况下，一个主机上只运行一个Task。

在分布式深度学习框架中，我们一般把Job划分为Parameter Server和Worker：Parameter Job是管理参数的存储和更新工作；Worker Job是用来运行ops。如果参数的数量太大，一台主机处理不了，这就要需要多个Tasks。

2.2 分布式TensorFlow模式

- In-graph 模式

将模型的计算图的不同部分放在不同的主机上执行。

In-graph模式和单机多GPU模型有点类似。还是一个小明算加法，但是算乘法的就可以不止是他们一个教室的小华，小红，小亮了。可以是其他教师的小张，小李。

In-graph模式，把计算已经从单机多GPU，已经扩展到了多机多GPU了，不过数据分发还在一个节点。这样的好处是配置简单，其他多机多GPU的计算节点，只要起个join操作，暴露一个网络接口，等在那里接受任务就好了。这些计算节点暴露出来的网络接口，使用起来就跟本机的一个GPU的使用一样，只要在操作的时候指定tf.device("/job:worker/task:n")，就可以向指定GPU一样，把操作指定到一个计算节点上计算，使用起来和多GPU的类似。但是这样的坏处是训练数据的分发依然在一个节点上，要把训练数据分发到不同的机器上，严重影响并发训练速度。在大数据训练的情况下，不推荐使用这种模式。

- Between-graph 模式

数据并行，每台主机使用完全相同的计算图。

Between-graph模式下，训练的参数保存在参数服务器，数据不用分发，数据分片的保存在各个计算节点，各个计算节点自己算自己的，算完了之后，把要更新的参数告诉参数服务器，参数服务器更新参数。这种模式的优点是不用训练数据的分发了，尤其是在数据量在TB级的时候，节省了大量的时间，所以大数据



2.3 同步更新和异步更新

- in-graph模式和between-graph模式都支持同步和异步更新。
- 在同步更新的时， 每次梯度更新，都要等所有分发出去的数据计算完成后，返回结果之后，把梯度累加算了均值之后，再更新参数。这样的好处是loss的下降比较稳定， 但是这个的坏处也很明显， 处理的速度取决于最慢的那个分片计算的时间。
 - 在异步更新时， 所有的计算节点，各自算自己的， 更新参数也是自己更新自己计算的结果， 这样的优点就是计算速度快， 计算资源能得到充分利用，但是缺点是loss的下降不稳定， 抖动大。

在数据量较小的情况下， 各个节点的计算能力比较均衡， 推荐使用同步模式；在数据量很大情况下，各个主机的计算性能参差不齐的情况下，推荐使用异步的方式。

3. 应用实例

tensorflow官方有个分布式tensorflow的文档，但是例子没有完整的代码， 这里写了一个最简单的可以跑起来的例子，供大家参考，这里也傻瓜式给大家解释一下代码，以便更加通俗的理解。本文的实例——[基于分布式Tensorflow的手写字体识别](#)，都加了代码中文注释，更加通俗易懂。

3.1 运行步骤

ps节点执行：

AI生成项目 登录复制

```
1 # python distributed.py --job_name=ps --task_index=0
```

worker1节点执行：

AI生成项目 登录复制

```
1 # python distributed.py --job_name=worker --task_index=0
```

worker2 节点执行：

AI生成项目 登录复制

```
1 # python distributed.py --job_name=worker --task_index=1
```

3.2 实验代码

AI生成项目 登录复制

```
1 # encoding:utf-8
2 import math
3 import tempfile
4 import time
5 import tensorflow as tf
6 from tensorflow.examples.tutorials.mnist import input_data
7
8 flags = tf.app.flags
9 IMAGE_PIXELS = 28
```

展开

4. 参考文章

<http://blog.csdn.net/luodongri/article/details/52596780>
<http://blog.csdn.net/u012436149/article/details/53140869>
<http://blog.csdn.net/stdcoutzyx/article/details/51645396>
TensorFlow实战[黄文坚 唐源 著]